

Despliegue en Oracle Cloud

Matriz de brechas de investigación — registro detallado de la puesta en marcha del servidor. 16 de agosto de 2026.

Este documento recoge, paso por paso, todo lo hecho para llevar el sistema de una laptop a un servidor accesible desde internet: la creación de la cuenta y la infraestructura en Oracle Cloud, la configuración del servidor por terminal, la instalación de Docker y el despliegue de la aplicación. Incluye también los problemas que aparecieron por el camino y cómo se resolvieron, porque son la parte que no se puede reconstruir mirando el resultado final.

Resumen del estado

Concepto	Valor
Dirección pública	<code>https://147-224-233-59.sslip.io</code>
Estado	Funcionando, accesible desde internet
Coste	0 — nivel Always Free, sin subida a cuenta de pago
Cifrado	HTTPS con certificado de Let's Encrypt, renovación automática
Arranque automático	Comprobado con un reinicio real
Copias de seguridad	Diarias a las 03:15, se conservan 7 días
Modo del modelo	mock (simulado), pendiente de pasar a real

1. Cuenta e infraestructura en Oracle Cloud

1.1 Cuenta

Se creó una cuenta gratuita en oracle.com/cloud/free. Oracle exige verificar la identidad con una tarjeta (débito con logo Visa, Mastercard, Amex o Discover; no admite virtuales ni de prepago). La verificación consiste en una retención de aproximadamente un dólar que se devuelve sola.

Campo	Valor elegido	Motivo
Nombre de cuenta	nubelab007	Neutro y reutilizable. Oracle advierte de que aparece en la URL y no se puede cambiar.
Región de origen	Chile West (Valparaíso)	No se puede cambiar después. Se eligió Valparaíso por ser más nueva y menos saturada.

Sobre el coste: la cuenta combina una prueba de 30 días con 300 USD de crédito y los recursos **Always Free**, que no caducan. Al terminar la prueba, si no se pulsa el botón de subir a cuenta de pago, lo gratuito sigue funcionando y lo demás deja de estar disponible. No hay cobro automático por uso ni por número de usuarios.

1.2 Red virtual

Este fue el primer punto donde hubo que rectificar. El formulario de creación de la instancia permite crear la red sobre la marcha, pero con opciones recortadas: el interruptor de dirección IP pública quedaba bloqueado y esa vía puede no crear la **pasarela de internet**. Sin ella la máquina arranca pero queda inaccesible desde fuera, sin SSH y sin web, y habría que borrarla y rehacerla.

Se creó la red por separado con el asistente completo, en **Networking** → **Virtual Cloud Networks** → **Actions** → **Start VCN Wizard** → **Create VCN with Internet Connectivity**.

Recurso creado	Detalle
VCN	vcn-matriz — CIDR 10.0.0.0/16
Subred pública	public subnet-vcn-matriz (regional)
Subred privada	private subnet-vcn-matriz
Pasarela de internet	Internet gateway-vcn-matriz
Pasarela NAT	NAT gateway-vcn-matriz
Service gateway	Acceso a servicios de Oracle sin salir a internet
DNS	vcnmatriz.oraclevcn.com

Con la red ya existente, al volver al formulario de la instancia y seleccionar la subred pública, el interruptor de IP pública se activó sin problema.

1.3 Máquina virtual

Campo	Valor
Nombre	instance-20260816-0335
Imagen	Canonical Ubuntu 24.04 (aarch64)
Forma	VM.Standard.A1.Flex — etiqueta Always Free-eligible
Procesador	4 OCPU ARM Ampere Altra a 3,0 GHz
Memoria	24 GB
Red	4 Gbps
Disco de arranque	46,6 GB, cifrado en tránsito activado
Dominio de disponibilidad	AD-1, fault domain FD-1
IP pública	147.224.233.59
Usuario del sistema	ubuntu

Los 4 OCPU y 24 GB son exactamente el máximo que el nivel Always Free concede por cuenta para máquinas ARM: se está usando todo lo gratuito sin pasarse. El estimador de costes de Oracle muestra un precio (unos 7,40 PEN al mes) porque no descuenta el nivel gratuito; lo que manda es la etiqueta *Always Free-eligible* junto a la forma.

No apareció el error *out of host capacity*, habitual en el nivel gratuito. De haber aparecido, la salida era reintentar en otro momento o bajar a 2 OCPU y 12 GB.

1.4 Claves de acceso

Se generaron con la opción **Generate a key pair for me** y se descargaron ambas. La privada no se puede volver a descargar: sin ella no hay forma de entrar al servidor.

Archivo	Ubicación actual
ssh-key-2026-08-16.key (privada)	D:\Claves Oracle\
ssh-key-2026-08-16.key.pub (pública)	D:\Claves Oracle\

1.5 Reglas de red

En la lista de seguridad por defecto de la red se añadieron dos reglas de entrada. Las tres primeras ya venían de fábrica.

Origen	Protocolo	Puerto	Para qué
0.0.0.0/0	TCP	22	SSH (venía por defecto)
0.0.0.0/0	ICMP	3, 4	Diagnóstico de red (por defecto)
10.0.0.0/16	ICMP	3	Diagnóstico interno (por defecto)
0.0.0.0/0	TCP	80	HTTP — añadida
0.0.0.0/0	TCP	443	HTTPS — añadida, para cuando haya certificado

2. Configuración del servidor por terminal

2.1 Primera conexión y permisos de la clave

El primer intento de conexión falló con *UNPROTECTED PRIVATE KEY FILE*. Windows deja los archivos nuevos legibles por otros usuarios del equipo, y SSH se niega a usar una clave privada en esas condiciones.

```
ssh -i "D:\Claves Oracle\ssh-key-2026-08-16.key" ubuntu@147.224.233.59
```

El primer arreglo, quitar la herencia de permisos, no bastó: los permisos no eran heredados sino puestos directamente sobre el archivo. Hubo que retirarlos por nombre.

```
icacls "D:\Claves Oracle\ssh-key-2026-08-16.key" /remove:g "NT AUTHORITY\Usuarios  
autenticados" /remove:g "BUILTIN\Usuarios" /remove:g "BUILTIN\Administradores"
```

Tras esto el archivo quedó accesible solo por el usuario y por SYSTEM, y la conexión funcionó.

2.2 Actualización del sistema

```
sudo apt-get update && sudo apt-get upgrade -y
```

Se aplicaron 64 actualizaciones, 60 de ellas de seguridad. El sistema pidió reiniciar por una actualización del núcleo, y se reinició con `sudo reboot`.

2.3 Cortafuegos interno

Este paso es el que más se olvida y el más difícil de diagnosticar después. Las imágenes de Ubuntu de Oracle traen **iptables bloqueando los puertos 80 y 443 dentro de la propia máquina**, aunque estén abiertos en el panel de Oracle. El síntoma sería una web que no responde sin ningún error visible.

```
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 80 -j ACCEPT  
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 443 -j ACCEPT  
sudo netfilter-persistent save
```

El último comando hace que las reglas sobrevivan a los reinicios; sin él se perderían al apagar la máquina.

2.4 Instalación de Docker

Desde el repositorio oficial de Docker, no desde el de Ubuntu, que suele ir por detrás.

```
sudo install -m 0755 -d /etc/apt/keyrings && \ sudo curl -fsSL  
https://download.docker.com/linux/ubuntu/gpg \ -o /etc/apt/keyrings/docker.asc && \ sudo chmod  
a+r /etc/apt/keyrings/docker.asc
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]  
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME")  
stable" | \ sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt-get update && sudo apt-get install -y docker-ce docker-ce-cli \ containerd.io  
docker-buildx-plugin docker-compose-plugin
```

```
sudo usermod -aG docker ubuntu
```

El último comando permite usar Docker sin escribir `sudo` delante de cada orden, pero solo surte efecto al volver a iniciar sesión. Se comprobó la instalación con `docker run --rm hello-world`, que descargó y ejecutó una imagen ARM nativa.

Componente	Versión instalada
Docker Engine	29.7.2
Docker Compose	v5.4.0

3. Despliegue de la aplicación

3.1 Descarga del proyecto

```
git clone https://github.com/Carlod007/capstone-matriz.git cd capstone-matriz git checkout CarlosDev
```

El cambio de rama es imprescindible y fue el segundo tropiezo: `git clone` descarga la rama principal, **main**, que va unos treinta commits por detrás y no contiene los archivos de Docker. El primer intento de levantar el sistema falló con *no configuration file provided: not found*.

3.2 Variables de entorno

Se creó el archivo de configuración generando los secretos **en el propio servidor**, de modo que la contraseña de la base y el secreto de sesión no pasaron por ningún sitio intermedio ni quedaron en el historial de nadie.

```
cat > .env <<EOF MYSQL_PASSWORD=$(openssl rand -hex 24) MYSQL_BASE=capstone
JWT_SECRETO=$(openssl rand -hex 32) JWT_HORAS=8 REGISTRO_ABIERTO=false GEMINI_MODE=mock
GEMINI_API_KEY= CORS_ORIGENES=http://147.224.233.59 VITE_API_BASE=/api PUERTO_BACKEND=8000
PUERTO_FRONTEND=80 EOF
```

Esos valores son los de aquel momento. Con el paso a Caddy (§3.6), `PUERTO_FRONTEND` se sustituyó por `PUERTO_HTTP` y `PUERTO_HTTPS`, y se añadió `DOMINIO`. La configuración vigente está en `.env.example`.

Se arrancó deliberadamente en modo **mock**: primero comprobar que todo levanta, y solo después poner la clave real de la API. Así un fallo de configuración no consume cuota.

3.3 Construcción y arranque

```
docker compose up -d --build
```

La construcción tardó unos dos minutos y levantó cinco servicios:

Servicio	Función	Estado esperado
mysql	Base de datos, con volumen propio	healthy
migraciones	Aplica el esquema con Alembic y termina	exited (correcto)
backend	API en FastAPI	up
trabajador	Procesa la cola de análisis	up
frontend	Servidor web y proxy hacia la API	up

Las migraciones corren como servicio aparte y los demás esperan a que termine. El esquema se creó solo, de la revisión 0001 a la 0005; después se añadió la 0006, que limpia los ROUGE que se habían guardado como cero cuando no eran aplicables.

3.4 El proxy inverso: tercer tropiezo

Nota de lectura: este apartado cuenta lo que se hizo aquel día, con nginx. Poco después se sustituyó por Caddy para conseguir el certificado de forma automática (§3.6). Se conserva el relato porque el problema y su razonamiento siguen siendo los mismos; solo cambió la herramienta que lo resuelve.

Con todo levantado, la página cargaba pero el inicio de sesión respondía *No se pudo contactar con el servidor*. La causa: el navegador llamaba directamente a `http://147.224.233.59:8000`, y ese puerto no está abierto — solo lo están el 22, el 80 y el 443.

Abrir el puerto 8000 habría funcionado, pero deja el backend expuesto a internet y obliga a mantener una lista de orígenes permitidos. La solución correcta fue poner nginx a hacer de intermediario, de modo que la

aplicación y su API compartan origen:

Cambio	Efecto
VITE_API_BASE pasa a CORS	Dirección/fuente: el navegador pide al mismo servidor que le sirvió la página. CORS deja de inter
nginx: location /api/ → backend:8000	El proxy traduce /api/proyectos a /proyectos. La barra final es la que quita el prefijo.
El puerto del backend solo escucha en 127.0.0.1	Accesible para depurar desde el servidor, invisible desde internet.
client_max_body_size 64m	nginx acepta un mega por defecto. Sin esto, subir un PDF devolvería un error 413 que no mencio
proxy_read_timeout 300s	Generar el estado del arte puede pasar de los sesenta segundos que nginx espera por defecto.

Este cambio se hizo en el repositorio, se subió, y en el servidor se aplicó con:

```
git pull sed -i 's|^VITE_API_BASE=.*|VITE_API_BASE=/api|' .env docker compose up -d --build frontend
```

3.5 Cuenta de acceso

La base del servidor es independiente de la de la laptop: empieza vacía, sin cuentas ni proyectos. La primera cuenta se crea desde la terminal porque el alta por web está cerrada a propósito.

```
docker compose exec backend python crear_cuenta.py
```

3.6 HTTPS con certificado

Faltaba lo más importante: la contraseña de inicio de sesión viajaba sin cifrar. Los certificados gratuitos exigen un dominio y una dirección IP suelta no sirve.

Se resolvieron las dos cosas a la vez. Como dominio se usó **sslip.io**, un servicio que convierte cualquier IP en un nombre sin registrarse en ningún sitio: la dirección pasó a ser 147-224-233-59.sslip.io. Y para el certificado se sustituyó nginx por **Caddy**.

El motivo del cambio no es potencia sino modos de fallo. Caddy pide, instala y renueva los certificados por su cuenta. Con nginx habría que añadir certbot, montar volúmenes de certificados y programar la renovación; y una renovación que falla, falla en silencio: uno se entera cuando el navegador empieza a avisar de un certificado caducado.

Aspecto	Cómo queda
Configuración	Un archivo, frontend/Caddyfile, de una pantalla
Modo	La variable DOMINIO decide: con dominio, HTTPS; sin él, HTTP sin cifrar para desarrollo
Certificados	En un volumen propio. Si se borrara, Caddy los volvería a pedir y Let's Encrypt limita cuántas veces por semana
Puerto 80	Se publica aunque se use HTTPS: por ahí comprueba Let's Encrypt que el dominio es de uno
Renovación	Automática, cada dos meses, sin intervención
Puertos	PUERTO_HTTP y PUERTO_HTTPS sustituyen a PUERTO_FRONTEND

Caddy asume además todo lo que hacía nginx —servir la interfaz, resolver las rutas profundas del frontend, hacer de intermediario en /api, el límite de 64 MB por subida y la espera larga para el estado del arte—, de modo que el apartado 3.4 sigue describiendo el comportamiento vigente aunque el programa sea otro. nginx.conf ya no existe; su equivalente es frontend/Caddyfile.

En el servidor se aplicó con:

```
git pull # en el .env: DOMINIO=147-224-233-59.sslip.io, PUERTO_HTTP=80, PUERTO_HTTPS=443 docker compose up -d --build frontend
```

El registro de Caddy confirmó *certificate obtained successfully* y la comprobación devolvió HTTP/2 200.

3.7 Arranque automático y copias de seguridad

Arranque automático. Los servicios llevan `restart: unless-stopped` y Docker arranca con el sistema, pero eso había que comprobarlo y no suponerlo. Se reinició el servidor de verdad y la aplicación volvió sola.

Copias de seguridad. Desde que el sistema vive en un servidor, los datos ya no están también en la laptop: perder el volumen de MySQL sería perderlo todo. El script `respaldar.sh` vuelca la base comprimida y conserva los últimos siete días.

Decisión	Motivo
Credenciales desde el <code>.env</code>	No tenerlas escritas en dos sitios, que acabarían desincronizados
<code>mysqldump --single-transaction</code>	El volcado se toma de una instantánea coherente sin bloquear la base: la aplicación sigue atendiendo m
Comprobación del tamaño	Un volcado fallido deja un <code>.gz</code> válido pero casi vacío, que pasaría por bueno hasta el día que hiciera falta
Fuera del repositorio	Contienen los correos y los hashes de contraseña de las cuentas

```
15 3 * * * cd /home/ubuntu/capstone-matriz && ./respaldar.sh >> respaldos/registro.log 2>&1
```

4. Problemas encontrados, en orden de aparición

Se recogen aquí porque son lo que no se puede deducir mirando el resultado final, y porque cuatro de los cinco habrían dejado el sistema en un estado que parece funcionar pero no funciona.

Problema	Causa	Solución
El interruptor de IP pública no se activaba desde el formulario de la instancia	Se activaba desde el formulario de la instancia	Creada por CarlosDev, con el asistente de configuración de la instancia
UNPROTECTED PRIVATE KEY FILE	Windows deja los archivos legibles por otros usuarios, por lo que se debe cambiar los permisos de los archivos	Se cambian los permisos de los archivos de Docker
no configuration file provided	git clone descarga main, que va treinta commits por detrás de CarlosDev	git clone de CarlosDev
No se pudo contactar con el servidor	El navegador llamaba al puerto 8000, cerrado por el firewall	Se cambia el puerto a 8080 en /api, mismo origen
Puertos 80 y 443 abiertos pero las imágenes de Docker bloquean esos puertos	Las imágenes de Docker bloquean esos puertos	Se desactiva el firewall de la máquina

5. Referencia rápida

5.1 Conectarse al servidor

```
ssh -i "D:\Claves Oracle\ssh-key-2026-08-16.key" ubuntu@147.224.233.59 cd capstone-matriz
```

Cada conexión deja en la carpeta personal, así que el cd hace falta siempre.

5.2 Operaciones habituales

Para qué	Comando
Ver el estado	docker compose ps
Ver los registros	docker compose logs -f backend
Ver los del trabajador	docker compose logs -f trabajador
Reiniciar todo	docker compose restart
Parar	docker compose down
Arrancar	docker compose up -d
Actualizar tras un cambio	git pull && docker compose up -d --build
Crear una cuenta	docker compose exec backend python crear_cuenta.py
Más trabajadores a la vez	docker compose up -d --scale trabajador=3

5.3 Datos de la instalación

Concepto	Valor
Aplicación	https://147-224-233-59.sslip.io
Servidor	147.224.233.59, usuario ubuntu
Carpeta del proyecto	~/capstone-matriz
Rama	CarlosDev
Clave privada	D:\Claves Oracle\ssh-key-2026-08-16.key
Datos que persisten	Volúmenes capstone-matriz_datos_mysql, capstone-matriz_pdfs y los de Caddy con los certificados. Sobre

Respaldos	~/capstone-matriz/respaldos/, diarios, 7 días
Servidor web	Caddy (frontend/Caddyfile). nginx.conf ya no existe

5.4 Advertencias vigentes

Está en modo simulado. Los textos que genere son de relleno. Para resultados reales hay que poner la clave de Gemini en el `.env` del servidor, cambiar `GEMINI_MODE` a `real` y reiniciar los contenedores.

La cuota es de la clave, no del usuario. Son 20 análisis diarios para toda la instancia. Por eso el alta de cuentas por web está cerrada (`REGISTRO_ABIERTO=false`).

La base del servidor es independiente de la de la laptop: son dos instalaciones separadas, cada una con sus cuentas y sus proyectos.

Los respaldos viven en la misma máquina que la base. Protegen de un borrado accidental o de una migración que salga mal, pero no de perder la instancia. Conviene bajarlos de vez en cuando con `scp`. Y un respaldo que nunca se ha restaurado no es un respaldo: el procedimiento para comprobarlo sobre una base aparte está al final de `respalda.sh`.